

СПИСЪК ОТ КЛАСОВЕ

Досега сме работили със списък от прости типове (int, string...)

```
List<int> numbers = new List<int>() { 1, 2, 3 };
```

Научихме се и да създаваме класове (шаблони), които съдържат различни прости типове като :

```
public class Student
{
    public string Name { get; set; }
    public int Age { get; set; }
}
```

Затова обектите, които се създават като членове на клас, всъщност са образно сложен-съставен тип.

Т.е. прости типове са int, string, double...

сложни са класовете.

Можем да работим със списък от класове, което позволява по-добра структура на данните.

В задачите от днес следваме следваме стъпките:

- 1) Създаване на клас
- 2) Създаване на списък
- 3) Добавяне на елементи
- 4) Отпечатване

1 зад: Нека Създадем клас Student по познатия начин:

- а) без конструктор
- б) с конструктор

а) без конструктор

```
class Student
{
    3 references
    public string Name { get; set; }
    3 references
    public int Age { get; set; }

    /*public Student(string name, int age) - без конструктор
    {
        Name = name;
        Age = age;
    }*/
    3 references
    public void DisplayInfo()
    {
        Console.WriteLine($"Student {Name} is {Age} years old.");
    }
}
```

Ако нямаме конструктор, всеки път при създаване на нов обект, трябва да пишем свързането
Name = ... Age = ...

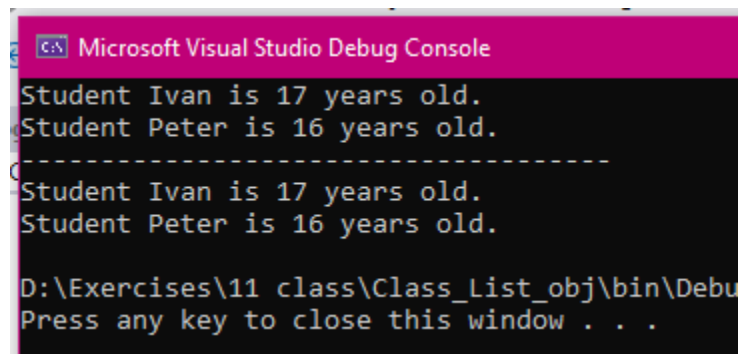
```
static void Main(string[] args)
{
    1 List<Student> students = new List<Student>(); // веднъж създаваме списъка
    string name = "Ivan";
    int age = 17;

    2 Student s1 = new Student { Name = name, Age = age }; // Вместо конструктор
    students.Add(s1);
    s1.DisplayInfo();

    Student s2 = new Student { Name = "Peter", Age = 16 };
    students.Add(s2);
    s2.DisplayInfo();
    Console.WriteLine(new string('-', 37));

    foreach (Student student in students)
    {
        student.DisplayInfo();
    }
}
```

Резултат:



```
Microsoft Visual Studio Debug Console
Student Ivan is 17 years old.
Student Peter is 16 years old.
-----
Student Ivan is 17 years old.
Student Peter is 16 years old.

D:\Exercises\11 class\Class_List_obj\bin\Debug
Press any key to close this window . . .
```

б) – с конструктор

Нека направим почти идентичен клас Teacher, но с конструктор като свързващ метод:

```
class Teacher
{
    2 references
    public string Name { get; set; }
    2 references
    public int Age { get; set; }

    0 references
    public Teacher(string name, int age)
    {
        Name = name;
        Age = age;
    }

    0 references
    public void DisplayInfo()
    {
        Console.WriteLine($"Teacher {Name} is {Age} years old.");
    }
}
```

конструктор

В Main() само добавяме разделител за по-добра нагледност, след което следваме стъпките:

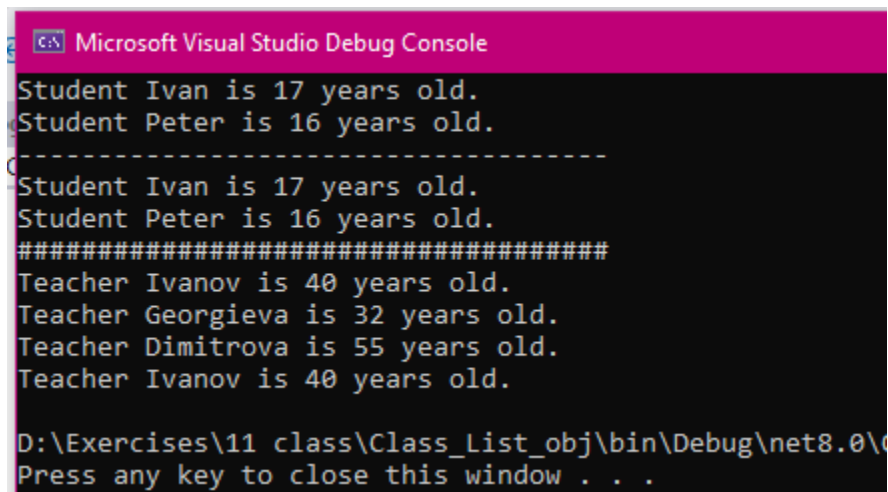
```
Console.WriteLine(new string('#', 37));
1 - създаване на списъка
List<Teacher> teachers = new List<Teacher>();
2 - добавяне на елементи
teachers.Add(new Teacher( "Ivanov", 40 ));
teachers.Add(new Teacher("Georgieva", 32));
teachers.Add(new Teacher("Dimitrova", 55));

foreach(Teacher teacher in teachers)
{
    teacher.DisplayInfo();
}

teachers[0].DisplayInfo(); достъпване
```

Вижда се колко по-кратък е кодът.

Резултат:



```
Microsoft Visual Studio Debug Console
Student Ivan is 17 years old.
Student Peter is 16 years old.
-----
Student Ivan is 17 years old.
Student Peter is 16 years old.
#####
Teacher Ivanov is 40 years old.
Teacher Georgieva is 32 years old.
Teacher Dimitrova is 55 years old.
Teacher Ivanov is 40 years old.

D:\Exercises\11 class\Class_List_obj\bin\Debug\net8.0\
Press any key to close this window . . .
```

```
internal class Program
{
    static void Main(string[] args)
    {
        List<Student> students = new List<Student>();

        string name = "Ivan";
        int age = 17;

        Student s1 = new Student { Name = name, Age = age };
        students.Add(s1);
        s1.DisplayInfo();

        Student s2 = new Student { Name = "Peter", Age = 16 };
        students.Add(s2);
        s2.DisplayInfo();
    }
}
```

```

        Console.WriteLine(new string('-', 37));

        foreach (Student student in students)
        {
            student.DisplayInfo();
        }

        Console.WriteLine(new string('#', 37));

        List<Teacher> teachers = new List<Teacher>();

        teachers.Add(new Teacher( "Ivanov", 40 ));
        teachers.Add(new Teacher("Georgieva", 32));
        teachers.Add(new Teacher("Dimitrova", 55));

        foreach(Teacher teacher in teachers)
        {
            teacher.DisplayInfo();
        }

        teachers[0].DisplayInfo();
    }
}
class Student
{
    public string Name { get; set; }
    public int Age { get; set; }

    public void DisplayInfo()
    {
        Console.WriteLine($"Student {Name} is {Age} years old.");
    }
}
class Teacher
{
    public string Name { get; set; }
    public int Age { get; set; }

    public Teacher(string name, int age) // ctor
    {
        Name = name;
        Age = age;
    }

    public void DisplayInfo()
    {
        Console.WriteLine($"Teacher {Name} is {Age} years old.");
    }
}

```